

9 [page Replacement Policy]

A system has RAM which can store four pages simultaneously to satisfy page requests. Write a Program to calculate percentage page hit and page fault if the system uses Optimal page replacement technique for the page references entered by the user i.e. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4,5 (No of Frames 3).

CODE:

```
GNU nano 8.7 optimal.c
#include <stdio.h>

int main()
{
    int pages[] = {1,2,3,4,1,2,5,1,2,3,4,5};
    int frames[3];
    int i,j,k,count=12,found,flag;
    int page_fault=0,page_hit=0,pos;

    for(i=0;i<3;i++)
        frames[i]=-1;

    for(i=0;i<count;i++)
    {
        found=0;

        for(j=0;j<3;j++)
        {
            if(frames[j]==pages[i])
            {
                found=1;
                page_hit++;
                break;
            }
        }

        if(found==0)
        {
            if(i<3)
                frames[i]=pages[i];
            else
            {
                int farthest=-1,replace=-1;

                for(j=0;j<3;j++)
                {
                    flag=0;
                    for(k=i+1;k<count;k++)
                    {
                        if(frames[j]==pages[k])
                        {
                            if(k>farthest)
                            {
                                farthest=k;
                            }
                        }
                    }
                }
            }
        }
    }
}
```

[Read 77 lines]

Alt-H Help	Alt-W Write Out	Alt-F Where Is	Alt-X Cut	Alt-E Execute	Alt-L Location	Ctrl-U Undo	Ctrl-A Set Mark	Ctrl-J To Bracket	Ctrl-B Previous
Alt-X Exit	Alt-R Read File	Alt-N Replace	Alt-U Paste	Alt-J Justify	Alt-G Go To Line	Ctrl-E Redo	Ctrl-M Copy	Alt-W Where Was	Ctrl-F Next

```
GNU nano 8.7 optimal.c
for(j=0;j<3;j++)
{
    flag=0;
    for(k=i+1;k<count;k++)
    {
        if(frames[j]==pages[k])
        {
            if(k>farthest)
            {
                farthest=k;
                replace=j;
            }
            flag=1;
            break;
        }
    }
    if(flag==0)
    {
        replace=j;
        break;
    }
    frames[replace]=pages[i];
}
page_fault++;

printf("\nFrames: ");
for(j=0;j<3;j++)
    printf("%d ",frames[j]);
}

printf("\n\nTotal Page Faults = %d",page_fault);
printf("\nTotal Page Hits = %d",page_hit);

printf("\nPage Fault Percentage = %.2f", (page_fault/(float)count)*100);
printf("\nPage Hit Percentage = %.2f", (page_hit/(float)count)*100);

return 0;
}
```

OUTPUT:

```
ASUS@Kalash-Laptop MINGW64 ~
$ nano optimal.c

ASUS@Kalash-Laptop MINGW64 ~
$ gcc optimal.c -o optimal

ASUS@Kalash-Laptop MINGW64 ~
$ ./optimal

Frames: 1 -1 -1
Frames: 1 2 -1
Frames: 1 2 3
Frames: 1 2 4
Frames: 1 2 4
Frames: 1 2 4
Frames: 1 2 5
Frames: 1 2 5
Frames: 1 2 5
Frames: 3 2 5
Frames: 4 2 5
Frames: 4 2 5

Total Page Faults = 7
Total Page Hits = 5
Page Fault Percentage = 58.33
Page Hit Percentage = 41.67
ASUS@Kalash-Laptop MINGW64 ~
$ |
```